

CS4234: Optimization Algorithms
Semester 1, AY 2026–27
School of Computing, NUS

Assignment 0: Prerequisite Review

Instructions

- This assignment is **completely optional**; there is **no need to submit** solutions.
- Its purpose is to help you assess your familiarity with the prerequisites for this course. If some of the material is unfamiliar, you are encouraged to review it before the course begins.
- Solutions will be posted separately, but do not look into the solutions before you solve them on your own. All the required ideas can be found in standard undergraduate algorithms textbooks and course references.
- For each algorithmic question, give a clear description or pseudocode, analyze the running time, and provide a concise proof of correctness.

Complexity and reductions

Question 1. Recall the complexity classes **P** and **NP**. Define **NP-hardness** and **NP-completeness**. What must be shown to prove that a decision problem is **NP-complete**?

Question 2. Assume that *Circuit Satisfiability* is **NP-hard**. Give a polynomial-time reduction from *Circuit Satisfiability* to *CNF-SAT*.

Hint. Introduce one variable x_k for the output of each gate k . For example, if gate k is an AND gate whose inputs are the outputs of gates i and j , encode $x_k \leftrightarrow (x_i \wedge x_j)$ by

$$(\neg x_k \vee x_i) \wedge (\neg x_k \vee x_j) \wedge (x_k \vee \neg x_i \vee \neg x_j).$$

Construct analogous formulas for OR and NOT gates, and require the circuit's output variable to be true.

Question 3. Give a polynomial-time reduction from *CNF-SAT* to *3-SAT*. Your construction should handle clauses with fewer than three literals as well as clauses with more than three literals.

Hint. Split a long clause into a chain of 3-literal clauses using fresh variables; pad short clauses appropriately.

Question 4. Consider the decision version of *MAX-2-SAT*: given a 2-CNF formula ϕ with m clauses and an integer k , where $1 \leq k \leq m$, decide whether some truth assignment satisfies at least k clauses. Prove that this problem is NP-complete.

Hint. Reduce from *3-SAT* using a constant-size 2-CNF gadget for each clause.

Question 5. Consider the following decision problems:

- *Independent Set*: given an undirected graph $G = (V, E)$ and an integer k , decide whether G has an independent set of size at least k ;
- *Vertex Cover*: given $G = (V, E)$ and k , decide whether G has a vertex cover of size at most k .

Prove that both problems are NP-complete. You may use the relation between independent sets and vertex covers.

Question 6. In the *Clique* decision problem, the input is an undirected graph $G = (V, E)$ and an integer k , and the question is whether G contains a clique of size at least k . Prove that this problem is NP-complete.

Hint. Use the complement graph and reduce from *Independent Set*.

Question 7. Fix a polynomial-time computable function $\alpha(n) \geq 1$. Consider the following promise version of *TSP*. The input is a complete undirected graph on n vertices with nonnegative edge costs and a threshold K , and it is promised that exactly one of the following holds:

1. there is a tour of cost at most K , or
2. every tour has cost at least $\alpha(n)K$.

The task is to distinguish the two cases. Give a polynomial-time reduction from *Hamiltonian Cycle* to this promise problem, to establish NP-hardness. Clearly specify the edge costs and prove that your constructed instance satisfies the promise.

Algorithmic prerequisites

Question 8. In the *Fractional Knapsack* problem, item i has value $v_i \geq 0$ and weight $w_i > 0$, the knapsack has capacity W , and any fraction of an item may be selected. Design a polynomial-time algorithm, prove its correctness, and analyze its running time.

Question 9. In the *0-1 Knapsack* problem, each item must be selected in its entirety or not selected at all. Assume that the item weights and the capacity W are positive integers, encoded in binary. Design a dynamic-programming algorithm with running time $O(nW)$, where n is the number of items. Explain how to recover an optimal set of items. Is this a polynomial-time algorithm in the input length? Why or why not?

Question 10. Design an $O(|V|)$ -time algorithm for *Maximum Independent Set* on a tree $T = (V, E)$. State the dynamic-programming subproblems and recurrence, and explain how to recover an optimal independent set.

Question 11. In *Weighted Interval Scheduling*, each job j has a start time s_j , a finish time f_j , and a nonnegative value v_j . Two jobs are compatible if one finishes no later than the other starts. Design an algorithm that finds a maximum-value set of pairwise compatible jobs in $O(n \log n)$ time. State the recurrence, explain any preprocessing, and show how to reconstruct an optimal schedule.

Question 12. In the *Minimum-Coin Change* problem, the input consists of positive integer denominations $c_1, \dots, c_n$ and a target value B . An unlimited number of coins of each denomination is available.

1. Give a dynamic-programming algorithm that uses the minimum possible number of coins to make value B , or reports that B cannot be made. Analyze its running time and determine whether it is polynomial in the binary input length.
2. Give an instance on which the greedy rule “repeatedly take the largest denomination that does not exceed the remaining amount” fails.

Question 13. In *Unweighted Interval Scheduling*, the goal is to select the maximum number of pairwise compatible intervals. Prove that repeatedly selecting the available interval with the earliest finish time is optimal. Your proof should use an exchange argument. Also give counterexamples showing that the rules “earliest start time first” and “shortest interval first” need not be optimal.

Question 14. There are n jobs to be processed nonpreemptively on a single machine. Job j has processing time $p_j > 0$ and deadline d_j . If it completes at time C_j , its lateness is $L_j = C_j - d_j$. The objective is to minimize the maximum lateness $L_{\max} = \max_j L_j$. Give a greedy algorithm and prove its optimality using an exchange argument. State its running time.

Question 15. Let $G = (V, E)$ be a connected undirected graph with real edge weights. State and prove the cut property for minimum spanning trees. Use it to justify the correctness of either Kruskal’s or Prim’s algorithm, and give an implementation and running-time analysis for your chosen algorithm.